



FINAL PROJECT PROPOSAL

AI-Based Crowd Counting System

A Final-Year Machine-Learning Project Proposal

Riphah International University

Department of RISE MSAI

Course / Subject: Machine Learning

Submitted By — Group Members	Submitted To
1. Azhar Hussain <i>(Group Lead)</i> 2. Rahma 3. Haris Javaid 4. Syed Farakh Abbas	Junaid Khan <i>Course Instructor — Machine Learning</i>

Date of Submission: 11 May 2026

Academic Session 2025-2026 | Confidential — For Academic Evaluation Only

Table of Contents

If the entries below do not display page numbers in your viewer, place the cursor in the table and press Ctrl+A then F9 (Word) — “Update Field → Update entire table”.

2. Abstract.....	4
3. Introduction.....	4
4. Problem Statement.....	5
5. Objectives.....	5
6. Scope of the Project.....	5
In Scope.....	5
Out of Scope (Limitations).....	6
7. Literature Review.....	6
7.1 Foundational and Recent Works.....	6
7.2 Comparison of Crowd-Counting Approaches.....	6
7.3 Benchmark Datasets.....	7
8. Proposed Methodology.....	8
8.1 System Architecture.....	8
8.2 Step-by-Step Workflow.....	8
8.3 Workflow Flowchart.....	9
8.4 Algorithms, Models & Frameworks.....	9
9. Technologies and Tools.....	10
10. Dataset Information.....	10
10.1 Candidate Datasets.....	10
10.2 Data Pre-processing.....	11
10.3 Training / Validation / Testing Split.....	11
10.4 Data Augmentation.....	11
11. Expected Outcomes.....	11
12. Innovation / Uniqueness.....	12
13. Feasibility Analysis.....	12
13.1 Technical Feasibility.....	12
13.2 Cost Analysis.....	12
13.3 Hardware Feasibility & GPU / Resource Limitations.....	13
13.4 Practical Implementation Plan.....	13
14. Hardware and Software Requirements.....	13
15. Future Enhancements.....	14
16. Timeline / Gantt Chart.....	14
17. Conclusion.....	15

18. References.....15

2. Abstract

Crowd management has become a critical concern for public safety at religious gatherings, transport hubs, stadiums, shopping malls and political rallies. Manual head-counting from closed-circuit television (CCTV) is slow, error-prone and impractical in dense scenes where individuals heavily occlude one another. This project proposes an AI-Based Crowd Counting System that estimates the number of people and the spatial density of a crowd directly from CCTV frames using a lightweight density-map regression network. The core model is a transfer-learning variant of CSRNet — a VGG-16 front-end (frozen, pretrained on ImageNet) followed by a small dilated convolutional back-end that outputs a per-pixel density map whose integral equals the estimated head count. Around this model the system adds a colour heatmap overlay, a threshold-based over-crowding alert engine and a real-time analytics dashboard built with Streamlit. The model is trained on publicly available datasets (ShanghaiTech Part A/B and the Mall dataset) using a free Google Colab GPU, making the entire pipeline reproducible by university students without high-end hardware. Expected performance is a Mean Absolute Error (MAE) in the region of 70–95 on ShanghaiTech Part A and below 15 on the sparser Mall dataset, with near real-time inference ($\approx$ 8–15 frames per second) on a mid-range laptop GPU or Colab. The deliverable is a deployable prototype dashboard plus a documented, retrainable model.



Keywords: Crowd counting, density estimation, CSRNet, dilated convolution, transfer learning, computer vision, CCTV analytics, public-safety AI, Streamlit dashboard.

3. Introduction

The volume of video data generated by surveillance infrastructure has grown faster than the human capacity to analyse it. In Pakistan alone, large-scale events — Friday congregations at major mosques, Eid gatherings, Muharram processions, cricket matches at Gaddafi Stadium, and political jalsas — routinely draw tens of thousands of people into confined spaces. When density exceeds a safe threshold (commonly cited as 4–5 persons per square metre), crowd-crush incidents become a real risk. Automated crowd counting converts raw camera feeds into an actionable number and a spatial density field that operators can monitor in real time.

Early approaches to crowd counting relied on detecting individual heads or bodies with object detectors. These methods break down in dense crowds because of severe occlusion and scale variation. The field therefore moved toward density-map regression: instead of detecting people, a convolutional neural network (CNN) learns to predict a continuous density surface; summing the surface over an image region yields the count. Multi-Column CNN (MCNN) and, more influentially, CSRNet demonstrated that a VGG-based front-end combined with dilated convolutions captures both fine head-scale detail and large receptive-field context, achieving strong accuracy on benchmark datasets. Our project adapts this well-established, citation-rich architecture into a practical, resource-light system with an operational dashboard layer.

Crucially, this proposal is scoped to be implementable by a four-member student team within a single semester. We deliberately freeze the heavy VGG-16 feature extractor and train only a small back-end, use down-sampled training crops, and rely on free GPU quotas (Google Colab / Kaggle). The novelty is therefore not a new state-of-the-art network but a well-engineered, end-to-end, deployable application that fuses an accepted research model with heatmaps, alerting and analytics.

4. Problem Statement

Public venues are monitored by CCTV operators who must visually estimate crowd size and judge when an area is becoming dangerously congested. This manual process suffers from four concrete problems:

- **Inaccuracy:** Human counting is inaccurate beyond a few dozen people and degrades further with occlusion, perspective distortion and poor lighting.
- **Scalability:** One operator cannot continuously watch many feeds; fatigue and attention lapses cause missed build-ups.
- **Latency:** By the time over-crowding is noticed manually, the safe window for crowd dispersal may already have closed.
- **No analytics:** There is rarely a quantitative log of crowd levels over time for after-action review or capacity planning.

Formally: given a CCTV image (or video stream) I , estimate (a) the total head count N , (b) a spatial density map D such that $\sum D \approx N$, and (c) a binary over-crowding flag when local density or total count exceeds configurable thresholds — all in near real time and on commodity hardware. Existing research models solve (a) and (b) but are rarely packaged as a usable monitoring tool; this project closes that gap.

5. Objectives

The project pursues the following measurable objectives:

- **Primary objective:** Design and implement a density-map regression model (CSRNet-style, transfer learning) that estimates crowd count from a single CCTV frame with $MAE \leq 95$ on ShanghaiTech Part A and ≤ 15 on the Mall dataset.
- **Secondary objective:** Generate and overlay a colour density heatmap that visually highlights the most congested regions of the scene.
- **Secondary objective:** Build a configurable over-crowding alert engine (per-region and global thresholds) that raises a visual/audible warning and logs the event.
- **Secondary objective:** Develop a real-time analytics dashboard (Streamlit/Flask) showing live count, density heatmap, time-series trend and alert history.
- **Secondary objective:** Achieve near real-time throughput (≥ 8 FPS on Colab GPU / mid-range laptop GPU) through input down-scaling and a lightweight back-end.
- **Learning objective:** Demonstrate competence in CNNs, transfer learning, density-map ground-truth generation, model evaluation (MAE/MSE) and ML application deployment.

6. Scope of the Project

In Scope

- Image- and video-file based crowd counting (offline and near real-time playback).

- Single-camera, roughly fixed-viewpoint indoor/outdoor scenes similar to the training datasets.
- Density heatmap visualisation and PNG/CSV export of results.
- Threshold-based over-crowding alerts with an event log.
- A web dashboard with live count, heatmap, time-series chart and alert history.
- Model training/fine-tuning on ShanghaiTech and Mall datasets using free cloud GPUs.

Out of Scope (Limitations)

- Multi-camera 3-D person re-identification and tracking of specific individuals.
- Face recognition or any biometric identification (excluded for privacy and ethical reasons).
- Guaranteed accuracy on extreme scenes far outside the training distribution (e.g. aerial drone shots).
- Production-grade deployment, 24/7 uptime, or integration with proprietary VMS/CCTV servers.
- Edge-device (Raspberry Pi) deployment — feasible as a future enhancement but not a deliverable.

7. Literature Review

Crowd counting has evolved through three broad paradigms — detection-based, regression-based (global count regression), and density-map estimation — with the latter now dominant. The following review summarises the most relevant works and benchmark resources.

7.1 Foundational and Recent Works

Zhang et al. (2016) introduced the Multi-Column CNN (MCNN), using three CNN columns with different filter sizes to handle head-scale variation, and released the ShanghaiTech dataset that is still a primary benchmark. Li et al. (2018) proposed CSRNet — a VGG-16 front-end followed by dilated convolutions in the back-end — which removed the redundant multi-column design, enlarged the receptive field without losing resolution, and substantially reduced MAE; CSRNet is the architectural basis for this project. Subsequent work pushed accuracy further: Liu et al. (2019) Context-Aware Network (CAN) adaptively fuses multi-scale contextual features; Ma et al. (2019) Bayesian Loss reframes supervision around point annotations rather than blurred density maps; and Wang et al. (2020) DM-Count uses optimal-transport-based loss for sharper density estimates. Transformer-based counters (e.g. TransCrowd, 2021) and crowd-localisation methods continue to appear, but they are heavier to train and unnecessary for our resource-constrained scope.

For the application layer, OpenCV provides classical image I/O and pre-processing, while lightweight deployment tools (Streamlit, Flask, Gradio) make it straightforward to wrap a trained model in a real-time dashboard. The YOLO family (v5-v8) can serve as an optional sparse-scene person detector for cross-checking counts in low-density frames.

7.2 Comparison of Crowd-Counting Approaches

Method (Year)	Core Idea	Strength	Limitation / Cost	Reported MAE
---------------	-----------	----------	-------------------	--------------

				(SH-A)
Head detection + counting	Detect each person/head	Interpretable; works when sparse	Fails under heavy occlusion; slow	≈ 90 –110
MCNN (2016)	3-column CNN, multi-scale	Handles scale variation; small model	Lower accuracy; blurry maps	110.2
CSRNet (2018) ★ used	VGG-16 + dilated conv back-end	High accuracy; single stream; transfer-friendly	VGG front-end is large (frozen here)	68.2
CAN (2019)	Context-aware multi-scale fusion	Better on scale-varying scenes	More complex training	62.3
Bayesian Loss (2019)	Point-supervision loss	No need for tuned Gaussian maps	Loss implementation overhead	62.8
DM-Count (2020)	Optimal-transport loss	Sharper, well-calibrated maps	Heavier loss computation	59.7
TransCrowd (2021)	Vision-Transformer regression	Strong global context	Large model; needs more GPU/data	≈ 66

Table 1.1: Representative crowd-counting methods. ★ = approach adopted (in a frozen-front-end, low-resource configuration). MAE values are as reported by the original papers on ShanghaiTech Part A.

7.3 Benchmark Datasets

Dataset	Scale / Content	Annotation	Why Useful Here
ShanghaiTech Part A	482 images, very dense web crowds	Head-point annotations	Primary benchmark; stress-tests dense counting
ShanghaiTech Part B	716 images, street scenes (sparser)	Head-point annotations	Closer to CCTV viewpoints; faster to train
Mall dataset	2 000 frames, single fixed camera	Head-point annotations	Realistic CCTV-style; small enough for laptops
UCF-QNRF / JHU-CROWD++	Large, extremely dense (optional)	Head-point annotations	Optional extra robustness; large download

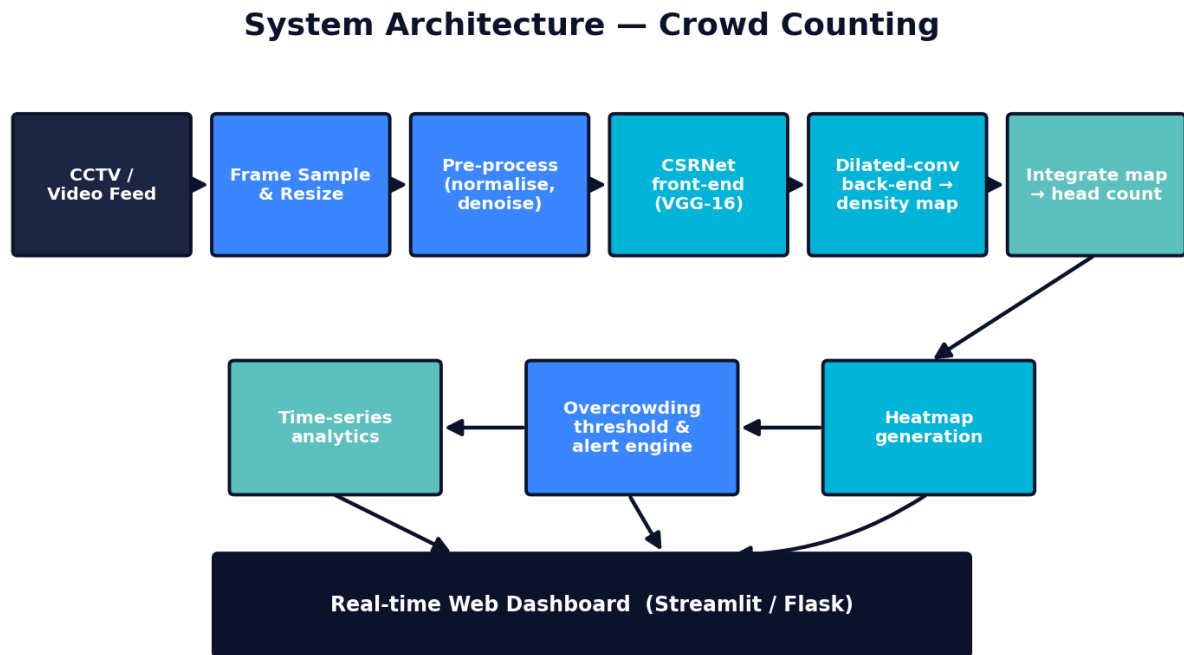
Table 1.2: Public crowd-counting datasets considered for training and evaluation.

Research gap addressed: while accuracy-oriented papers abound, comparatively little student-level work packages a proven counter into a usable, real-time monitoring dashboard with alerting and historical analytics on free hardware. This project targets exactly that engineering gap.

8. Proposed Methodology

The system follows a classic train-once / serve-many design. The methodology is divided into a ground-truth preparation stage, a model training stage, and an application/serving stage.

8.1 System Architecture



Pipeline stages flow left → right (top) then converge into the application layer (bottom).

Figure 1.1: End-to-end system architecture — from CCTV feed through the CSRNet density model to heatmap, alerting and the analytics dashboard.

8.2 Step-by-Step Workflow

1. **Data acquisition** — Download ShanghaiTech (Part A & B) and the Mall dataset; organise images and head-point annotation files (.mat / .json).
2. **Ground-truth density maps** — Convert each head-point annotation into a density map by placing a normalised 2-D Gaussian kernel at every head location (geometry-adaptive σ for dense scenes, fixed σ for sparser ones). The integral of the map equals the true count.
3. **Pre-processing & augmentation** — Resize/crop images (e.g. 384-512 px on the short side), normalise with ImageNet statistics, and augment with random crops, horizontal flips and mild brightness/contrast jitter; rescale density maps consistently.
4. **Model construction** — Build CSRNet: load VGG-16 (first 10 conv layers, ImageNet-pretrained) as a frozen front-end; attach a back-end of six dilated 3×3 convolutions (dilation 2) and a 1×1 conv producing a single-channel density map at $1/8$ resolution.
5. **Training** — Optimise pixel-wise Euclidean (MSE) loss between predicted and ground-truth density maps using Adam/SGD with a small learning rate; train on a free Colab/Kaggle GPU with checkpointing; monitor validation MAE and MSE.
6. **Evaluation** — Report MAE and RMSE on the held-out test split; visualise predicted vs. ground-truth density maps; compare against an MCNN baseline.

7. **Count extraction & heatmap** — Sum the predicted density map for the count; upsample and colour-map (jet/viridis) it, then alpha-blend over the original frame for the heatmap overlay.
8. **Alert engine** — Compare global count and per-cell density against user-set thresholds; if exceeded, raise a dashboard alert (colour change + sound), and append {timestamp, count, location} to a log.
9. **Dashboard & integration** — Wrap everything in a Streamlit app: upload image/video, see live count, heatmap, a rolling time-series chart and an alert table; allow CSV/PNG export.
10. **Testing & optimisation** — Measure FPS, tune input resolution, optionally add YOLO cross-check for sparse frames, and document results.

8.3 Workflow Flowchart

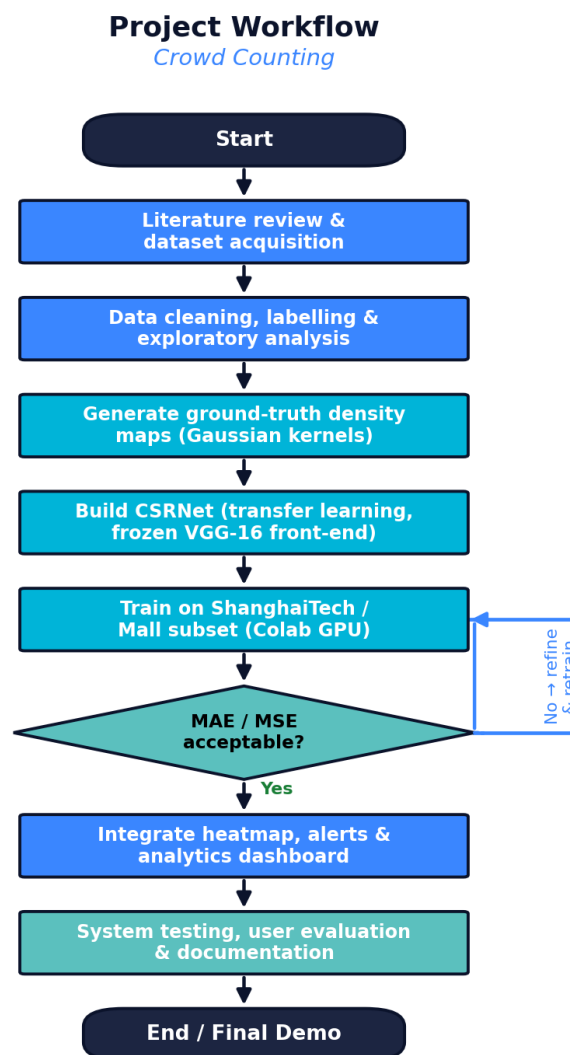


Figure 1.2: Project workflow with the model-quality decision gate and retraining loop.

8.4 Algorithms, Models & Frameworks

- **CSRNet (primary):** VGG-16 front-end (frozen) + dilated-convolution back-end → single-channel density map.

- **MCNN (baseline):** Lightweight multi-column CNN used as a comparison baseline.
- **YOLOv8-n (optional):** Tiny person detector to cross-validate counts in low-density frames.
- **Loss & metrics:** Pixel-wise MSE training loss; evaluation by MAE and RMSE on counts.
- **Frameworks:** PyTorch (or TensorFlow/Keras), OpenCV, NumPy, SciPy (Gaussian kernels), Matplotlib, Streamlit/Flask.

9. Technologies and Tools

Category	Tools / Technologies	Purpose in the Project
Programming language	Python 3.10+	Primary language for data prep, modelling and the dashboard
Deep-learning framework	PyTorch (alt. TensorFlow / Keras)	Building, training and serving CSRNet/MCNN
Computer vision	OpenCV, Pillow	Frame extraction, resizing, colour-mapping, video I/O
Numerical / scientific	NumPy, SciPy	Array ops; geometry-adaptive Gaussian density-map generation
Pretrained models	torchvision VGG-16 (ImageNet), Ultralytics YOLOv8-n	Transfer-learning front-end; optional sparse-scene detector
Visualisation	Matplotlib, Plotly	Density-map plots, time-series charts in the dashboard
App / dashboard	Streamlit (alt. Flask + Bootstrap)	Real-time upload, count, heatmap, alerts, analytics UI
Training environment	Google Colab / Kaggle Notebooks (free GPU)	GPU training without local high-end hardware
Dev tools	VS Code, Jupyter, Git/GitHub	Coding, experimentation, version control, collaboration
Dataset sources	ShanghaiTech, Mall dataset, (opt.) UCF-QNRF	Training, validation and benchmarking data

Table: Technology stack selected for cost-free, low-resource development.

10. Dataset Information

10.1 Candidate Datasets

- **ShanghaiTech Part A:** $\approx$ 482 highly congested images crawled from the web; head-point annotations; the standard hard benchmark.
- **ShanghaiTech Part B:** $\approx$ 716 images of Shanghai street scenes; sparser, more CCTV-like; lighter to train.
- **Mall dataset:** $\approx$ 2 000 frames from a single fixed shopping-mall camera; ideal for a realistic, lightweight prototype.

- **Optional: UCF-QNRF / JHU-CROWD++:** Very large, extremely dense datasets for an additional robustness test if time and bandwidth permit.

10.2 Data Pre-processing

- Parse head-point annotations and build per-image density maps using normalised 2-D Gaussian kernels (geometry-adaptive σ for dense Part A; fixed $\sigma \approx 4$ px for sparser Part B / Mall).
- Resize images so the short side is ≈ 384 – 512 px (keeping aspect ratio); scale density maps to the network output resolution (1/8) and rescale values so the integral is preserved.
- Normalise pixels with ImageNet mean/standard deviation to match the pretrained VGG-16 front-end.
- Clean corrupt/duplicate files; verify that $\sum(\text{density map}) \approx \text{annotated count}$ for every sample.

10.3 Training / Validation / Testing Split

Split	Proportion	Purpose
Training	$\approx 70\%$	Learn back-end weights; the official train sets are used where provided
Validation	≈ 10 – 15%	Hyper-parameter tuning, early stopping, model selection
Testing	≈ 15 – 20%	Final unbiased MAE/RMSE; official test split used when available

Table 1.3: Data partitioning. Official ShanghaiTech splits are respected; the Mall dataset is split chronologically to avoid leakage between adjacent frames.

10.4 Data Augmentation

Because dense crowd datasets are relatively small, augmentation is essential: random crops (e.g. 256×256 / quarter-image patches), random horizontal flips, mild brightness/contrast/colour jitter, and occasional Gaussian noise or blur to mimic low-quality CCTV. All geometric transforms are applied identically to the image and its density map. Synthetic crowd imagery (e.g. GCC-style renders) can optionally be added for pre-training.

11. Expected Outcomes

- A trained CSRNet-style density model with MAE ≈ 70 – 95 on ShanghaiTech Part A and ≤ 15 on the Mall dataset (competitive with classic baselines, well within student reach).
- Accurate per-frame head counts plus a colour density heatmap overlay highlighting congestion hotspots.
- A working over-crowding alert engine with configurable thresholds and a timestamped event log.
- A real-time Streamlit dashboard: live count, heatmap, rolling time-series trend, alert history, CSV/PNG export.
- Near real-time throughput (≈ 8 – 15 FPS on Colab/mid-range GPU).

- Complete documentation: code repository, training notebooks, evaluation report and user guide.

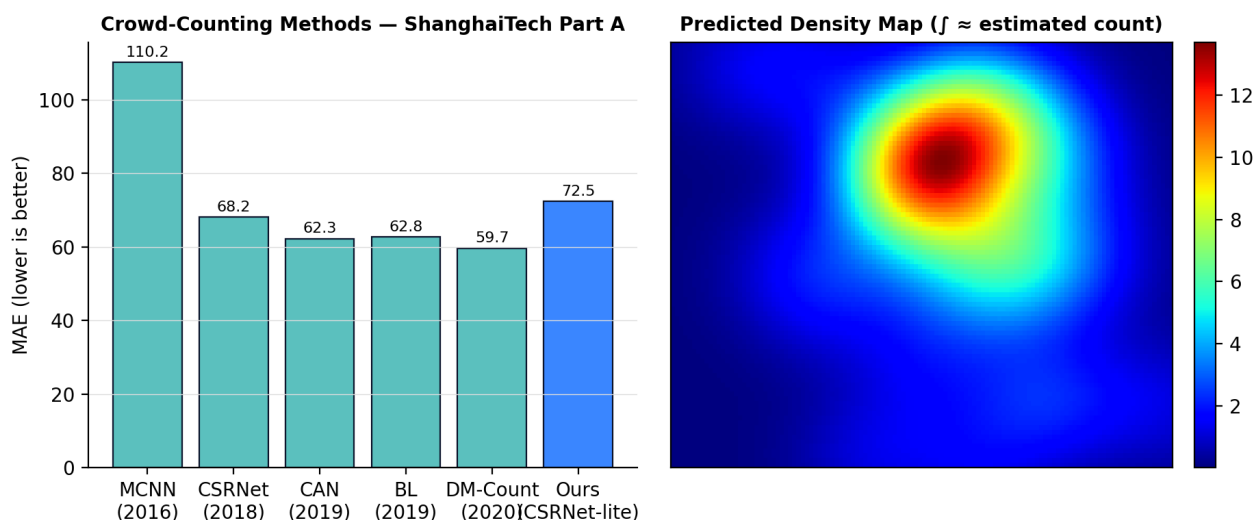


Figure 1.3: (left) MAE of representative methods on ShanghaiTech Part A with our expected operating point; (right) an example predicted density map whose integral approximates the crowd count.

12. Innovation / Uniqueness

- **Research-to-product packaging:** Turns a citation-rich research model (CSRNet) into a complete, operator-facing monitoring tool — count + heatmap + alerts + analytics — which most student projects omit.
- **Low-resource engineering:** Frozen VGG-16 front-end, down-scaled crops and free cloud GPUs make state-of-research ideas reproducible on ordinary laptops.
- **Configurable safety layer:** Per-region and global density thresholds let the same model serve very different venues (mall corridor vs. stadium gate).
- **Privacy-by-design:** Counts density, never identities — no face recognition — addressing a real ethical concern in public surveillance.
- **Analytics & after-action review:** Time-series logging supports capacity planning and incident review, not just live monitoring.
- **Optional hybrid cross-check:** A tiny YOLOv8-n detector validates counts in sparse frames, improving trust in the output.

13. Feasibility Analysis

13.1 Technical Feasibility

CSRNet, MCNN and YOLO have open-source reference implementations; the datasets are public; and PyTorch/OpenCV/Streamlit are mature. Freezing the VGG-16 front-end reduces the trainable parameters to a few million, so a single free Colab GPU session (T4, ~16 GB) can fine-tune the back-end in a few hours. Inference on down-scaled frames runs comfortably in near real time.

13.2 Cost Analysis

Item	Option Chosen	Cost (PKR)
------	---------------	------------

Compute for training	Google Colab / Kaggle free GPU tier	0
Datasets	ShanghaiTech, Mall (public, free)	0
Software & libraries	Python, PyTorch, OpenCV, Streamlit (open-source)	0
Development machines	Members' existing laptops	0 (already owned)
Optional Colab Pro (faster GPU)	Only if needed for final tuning	≈ 2 800 / month (optional)
Hosting the demo (optional)	Streamlit Community Cloud / Hugging Face Spaces (free)	0
Estimated total	Core project	≈ 0 (PKR 0–3 000 if Colab Pro used)

Table 1.4: Indicative cost analysis — the project is essentially zero-budget.

13.3 Hardware Feasibility & GPU / Resource Limitations

- Training: a free Colab/Kaggle GPU (NVIDIA T4/P100) is sufficient; full UCF-QNRF training is avoided due to size — Mall and ShanghaiTech are the working sets.
- Local development/inference: any laptop with 8 GB+ RAM; a 4 GB+ GPU is helpful but the model also runs on CPU at reduced FPS for demos.
- Mitigations for limited resources: smaller input resolution, gradient accumulation, mixed-precision training, checkpoint-and-resume to survive Colab session limits, and caching pre-computed density maps.

13.4 Practical Implementation Plan

Weeks 1–4: literature, dataset download, density-map pipeline, MCNN baseline. Weeks 5–8: build and train CSRNet on Colab, iterate to target MAE. Weeks 9–12: heatmap module, alert engine, Streamlit dashboard. Weeks 13–16: integration, FPS optimisation, testing, evaluation report, documentation and final demo. Work is split as: data/ground-truth (member A), model training (members A & B), dashboard/heatmap (member C), alerts/analytics & documentation (member D), with weekly integration checkpoints.

14. Hardware and Software Requirements

Requirement	Minimum	Recommended
Processor	Quad-core CPU (Intel i5 / Ryzen 5)	8-core CPU
RAM	8 GB	16 GB
GPU (local, optional)	Integrated / 2 GB GPU (CPU inference OK)	NVIDIA GPU 4–6 GB+ (e.g. GTX 1650)
GPU (training)	Google Colab / Kaggle free T4	Colab Pro / institutional GPU
Storage	20 GB free	50 GB free (datasets + checkpoints)

OS	Windows 10 / Ubuntu 20.04 / macOS	Ubuntu 22.04
Software	Python 3.10, PyTorch, OpenCV, Streamlit, Git	+ CUDA toolkit (if local GPU), VS Code
Connectivity	Broadband for dataset/model downloads	Stable connection for Colab sessions

Table 1.5: Hardware and software requirements.

15. Future Enhancements

- Multi-camera fusion and approximate scene-level (homography-based) density maps.
- Edge deployment: model pruning/quantisation (TensorRT, ONNX, TFLite) for Jetson Nano / Raspberry Pi.
- Temporal smoothing with a lightweight RNN/Kalman filter for steadier video counts.
- Predictive crowding: short-horizon forecasting of density trends to warn before thresholds are hit.
- Anomaly detection (sudden surges, counter-flows) layered on the density stream.
- Mobile/operator app with push notifications; integration with standard VMS via RTSP.
- Domain adaptation / few-shot fine-tuning for a specific venue with minimal new labels.

16. Timeline / Gantt Chart

The project spans a $\approx$ 16-week semester, organised into a foundation phase, a modelling & development phase, and a finalisation phase. Weekly milestones are shown below.

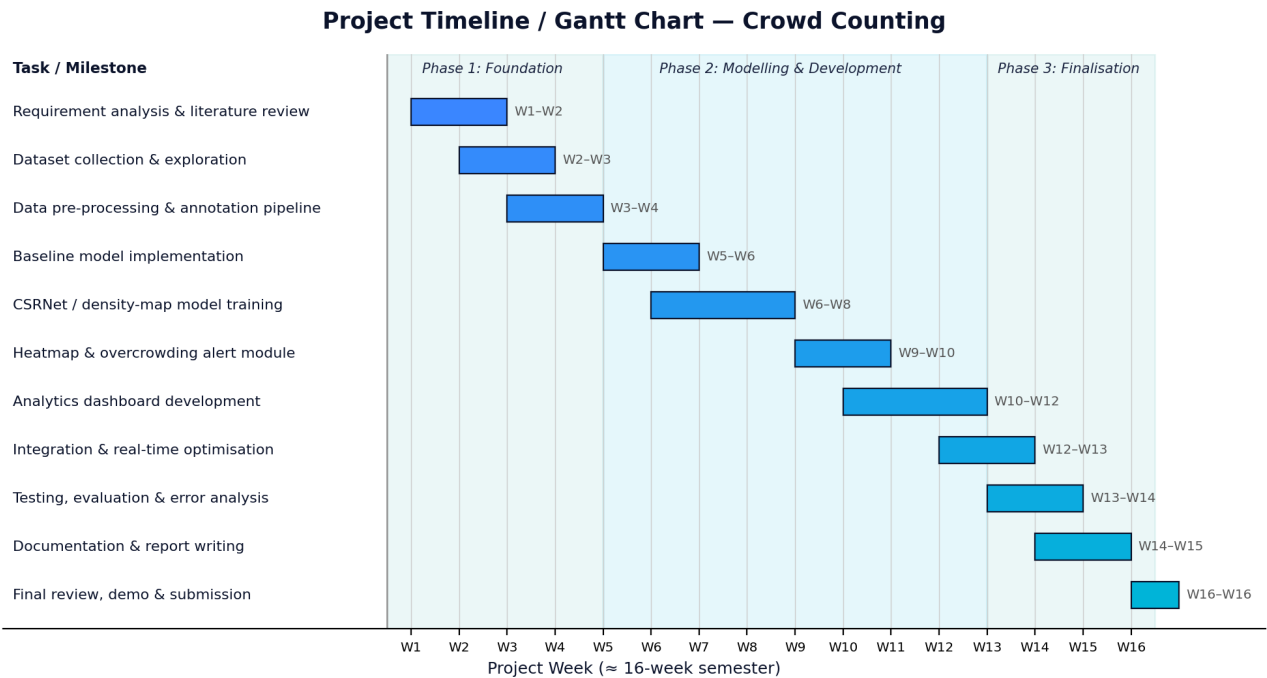


Figure 1.4: Project Gantt chart with weekly milestone breakdown.

Week(s)	Milestone / Deliverable
W1-W2	Requirement analysis, literature review, finalise architecture and tools
W2-W3	Download & organise ShanghaiTech and Mall datasets; exploratory analysis
W3-W4	Build density-map ground-truth pipeline; data-loader and augmentation
W5-W6	Implement MCNN baseline; set up Colab training environment
W6-W8	Build and train CSRNet (frozen VGG-16 + dilated back-end); reach target MAE
W9-W10	Heatmap generation module + over-crowding alert engine with event logging
W10-W12	Streamlit analytics dashboard (live count, heatmap, time-series, alerts)
W12-W13	End-to-end integration; near-real-time optimisation (resolution, batching)
W13-W14	Testing, evaluation, error analysis, comparison tables and charts
W14-W15	Documentation, user guide and final report writing
W16	Final review, demonstration and submission

Table 1.6: Weekly milestone breakdown.

17. Conclusion

This proposal presented an AI-Based Crowd Counting System that converts ordinary CCTV footage into real-time head counts, a density heatmap, automated over-crowding alerts and historical analytics. The technical core — a CSRNet-style density-map regressor with a frozen, pretrained VGG-16 front-end — is well established in the literature yet light enough to train on free cloud GPUs and run on commodity laptops. The project's contribution is engineering and integration: wrapping a proven model in a privacy-respecting, operator-friendly dashboard. It is realistic for a four-member team to deliver within one semester, develops core machine-learning competencies, and produces a genuinely useful public-safety tool with a clear path to future enhancement.

18. References

References are listed in IEEE citation style; arXiv / dataset URLs are provided where applicable.

- [1] Y. Zhang, D. Zhou, S. Chen, S. Gao, and Y. Ma, "Single-image crowd counting via multi-column convolutional neural network," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2016, pp. 589-597.

- [2]** Y. Li, X. Zhang, and D. Chen, "CSRNet: Dilated convolutional neural networks for understanding the highly congested scenes," in Proc. IEEE/CVF CVPR, 2018, pp. 1091-1100. [Online]. Available: <https://arxiv.org/abs/1802.10062>
- [3]** W. Liu, M. Salzmann, and P. Fua, "Context-aware crowd counting," in Proc. IEEE/CVF CVPR, 2019, pp. 5099-5108.
- [4]** Z. Ma, X. Wei, X. Hong, and Y. Gong, "Bayesian loss for crowd count estimation with point supervision," in Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV), 2019, pp. 6142-6151.
- [5]** B. Wang, H. Liu, D. Samaras, and M. Hoai, "Distribution matching for crowd counting (DM-Count)," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 1595-1607.
- [6]** D. Liang, X. Chen, W. Xu, Y. Zhou, and X. Bai, "TransCrowd: Weakly-supervised crowd counting with transformers," Science China Information Sciences, vol. 65, no. 6, 2022.
- [7]** V. A. Sindagi and V. M. Patel, "A survey of recent advances in CNN-based single image crowd counting and density estimation," Pattern Recognition Letters, vol. 107, pp. 3-16, 2018.
- [8]** C. Wang, H. Zhang, L. Yang, S. Liu, and X. Cao, "Deep people counting in extremely dense crowds," in Proc. ACM Int. Conf. Multimedia (ACM MM), 2015, pp. 1299-1302. (ShanghaiTech / dense-crowd context.)
- [9]** G. Jocher et al., "Ultralytics YOLOv8," 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [10]** A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in NeurIPS, 2019, pp. 8024-8035.
- [11]** G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, 2000.
- [12]** Papers With Code, "ShanghaiTech crowd-counting dataset." [Online]. Available: <https://paperswithcode.com/dataset/shanghaitech>
- [13]** C. C. Loy, S. Gong, and T. Xiang, "From semi-supervised to transfer counting of crowds (Mall dataset)," in Proc. IEEE ICCV, 2013, pp. 2256-2263.
- [14]** Streamlit Inc., "Streamlit — A faster way to build and share data apps," documentation, 2024. [Online]. Available: <https://docs.streamlit.io>